

Lessons Learned

Wilson Neira

This project taught me that building a real distributed system is much harder than just describing one on paper. At the beginning, I had a clear idea for Reliability Copilot, but my first version was still too simple. It could take in a failure event and return a result locally, but it did not yet reflect the AWS-based architecture I originally proposed. What went wrong early on was that my project idea was ahead of my actual implementation. I had the vision first, but I still needed to turn it into a real end-to-end system with queues, workers, storage, cloud deployment, and experiments. If I were doing this again, I would move faster from the idea stage into a small but realistic end-to-end version earlier.

One of the biggest lessons for me was that observability is not something you can just mention in a report and assume is there. Early on, it was easy for me to say I wanted CloudWatch metrics, logs, and system visibility, but much harder to build the project in a way that showed meaningful behavior. Once I got the AWS version working and started running the experiments, I understood much better why observability matters. Metrics like p95 latency, throughput, queue depth, CPU, memory, and logs are what made the system understandable. Without that, it would have just been code that seemed to work. With it, I could explain what was slowing down, what improved under scaling, and what changed when I tested protected versus unprotected failure handling.

Another lesson was that the AI part is only one piece of the system. At first, it is tempting to think the main value is just having the model or classifier say something useful about a failure. But what I learned is that the more important part is the system around it. The queue, the worker, the fallback handling, the storage, the AWS deployment, and the monitoring are what make it useful as a reliability platform instead of just a simple classifier demo. That connects directly to concepts from the course like decoupling, asynchronous processing, bottlenecks, fault tolerance, graceful degradation, and performance tradeoffs. I came away understanding much better that distributed systems are not mainly about writing endpoints. They are about controlling how work moves through the system and how the system behaves under pressure.

The most challenging technical part for me was moving from the earlier local prototype to the AWS version that actually matched my proposal. Once I started using ECS Fargate, SQS, DynamoDB, CloudWatch, and Terraform together, the project became much more realistic, but also much more difficult. I ran into problems with Docker, Go module versions, Terraform configuration, IAM restrictions in the Learner Lab, and getting the experiment pipeline to work cleanly from local runs to AWS runs. That was frustrating at times, but it also ended up being one of the valuable parts of the project. It forced me to debug across multiple layers: code, containers, cloud infrastructure, permissions, and experiment setup.

The experiments were also a big learning experience. In the bottleneck experiment, I saw that the system could stay up under load, but latency increased a lot as concurrency increased. That helped me understand that a system can still be working while performing badly. In the scaling

experiment, I saw that adding worker-side capacity improved stability and helped latency stay more controlled. That connected directly to course ideas about scaling not being magic, but instead being a tradeoff that helps only if the real bottleneck is in the right place. In the failure injection experiment, I saw that the protected version handled failure conditions better than the unprotected one. That was probably the clearest example of degradation from the course showing up in my own project. The protected version did not remove failure, but it reduced how badly the failure spread through the system.

Something else I learned is that cloud and lab environments add their own real-world complexity. My proposal assumed a clean AWS workflow with Terraform-managed infrastructure and auto scaling experiments, but in practice the Learner Lab permissions created limitations and extra troubleshooting. That taught me that real systems work often includes dealing with environment limits, permissions, and tooling issues that are not part of the ideal design.

If I were doing this project again, I would start with the AWS-aligned architecture earlier instead of growing from a simpler local version for so long. I would also make the experiment pipeline more standardized from the beginning, including naming conventions for results files, plotting scripts, and report-ready summaries, because a lot of my later work was spent cleaning up the experiment outputs and making them easier to explain. I would also try to get observability and experiment metrics in place earlier, because once those were working, the project became much easier to reason about.

Overall, I think this project shows real growth for me from the beginning of the course to the end. At the start, I was thinking more in terms of whether a system worked at all. By the end, I was thinking much more about tradeoffs, bottlenecks, queue buildup, resilience, measurements, and stakeholder value. I also feel that this project ties together a lot of what I want to do professionally: distributed systems, cloud infrastructure, reliability, and practical AI. So even though there were a lot of frustrating parts, I think it was a useful project I have done because it pushed me to build something closer to a real platform rather than just a simple assignment solution.